

# Final Projects

Hsi-Yu Schive (薛熙于)  
National Taiwan University

# General Guidelines and Requirements

1. Must parallelize your program by at least one of the following methods
  - a. OpenMP (minimum requirement)
  - b. MPI (get bonus point)
  - c. GPU (get extra bonus point)
2. Must provide convincing demonstration of the accuracy and performance of your program
3. Must use GitHub for collaborative development
  - a. Tutorial: <https://gitbook.tw/>
  - b. Use the Fork→Pull→Push→Pull Request→Merge workflow
  - c. Do NOT just upload the final code → must keep the development history on GitHub
4. Students per group: **3-4**
  - a. Inform the TA before **May 1**
5. Final presentation: **June 5, 12**
  - a. 25 mins presentation + 10 mins questions
  - b. All students are encouraged to ask ANY questions
6. Bonus points: *Surprise Me!!!*
7. Provide the following materials after the presentation: slides, a GitHub link (including usernames and corresponding Chinese names), and a task allocation table

# Project-I: High-order Data Reconstruction in Hydro

1. Implement at least one of the following data reconstruction methods
  - a. **Piecewise Parabolic Method (PPM)**  
<https://www.sciencedirect.com/science/article/pii/0021999184901438>  
<https://www.sciencedirect.com/science/article/pii/S0021999108001435>
  - b. **Piecewise Cubic Method (PCM)**  
<https://www.sciencedirect.com/science/article/pii/S0021999117302759?via%3Dihub>
  - c. **Weighted Essentially Non-Oscillatory (WENO)**  
<https://www.sciencedirect.com/science/article/pii/S0021999116301371>
  - d. **Gaussian Process (GP)**  
<https://link.springer.com/article/10.1007/s10915-017-0625-2>  
<https://www.sciencedirect.com/science/article/pii/S0021999119300099>
2. Compare the accuracy and performance with PLM
3. Bonus points
  - a. Implement the *characteristic tracing* step (described in some of these schemes, e.g., PPM & PCM) to improve the temporal resolution
  - b. Extend to 2D or even 3D
  - c. Support MHD
  - d. Add external and/or self-gravity

# Project-II: Multigrid Poisson Solver

1. Implement the multigrid method to solve the 2D Poisson equation
  - a. Compare the V and W cycles
  - b. Measure the performance scaling (i.e., wall-clock time vs. number of cells)
  - c. Compare the performance with SOR
2. Bonus points
  - a. Implement the *full multigrid method* (FMG) and compare with MG
3. Reference
  - a. Enzo:  
<https://github.com/enzo-project/enzo-dev/blob/master/src/enzo/MultigridSolver.C>
  - b. GAMER:  
[https://github.com/gamer-project/gamer/blob/master/src/SelfGravity/CPU\\_Poisson/CPU\\_PoissonSolver\\_MG.cpp](https://github.com/gamer-project/gamer/blob/master/src/SelfGravity/CPU_Poisson/CPU_PoissonSolver_MG.cpp)
  - c. Numerical Recipes, Chapter 20.6
  - d. Multigrid Techniques: 1984 Guide with Applications to Fluid Dynamics  
<https://epubs.siam.org/doi/book/10.1137/1.9781611970753>

# Project-III: Particle Mesh

1. Implement the particle mesh scheme in 3D
  - a. Poisson solver: both periodic and isolated boundary conditions with FFT
  - b. Mass deposition: NGP, CIC, and TSC → compare their accuracy
  - c. Orbit integration: KDK and DKD → compare their accuracy with variable time-steps
  - d. Measure the performance scaling (i.e., wall-clock time vs. number of cells/particles)
  - e. Validate the momentum conservation
2. Bonus points
  - a. Implement the *RK4 and/or Hermite* scheme for orbit integration
  - b. Individual time-step integration (see the reference of Hermite integration below)
3. Reference
  - a. Computer Simulation Using Particles by Hockney and Eastwood:  
<https://www.amazon.com/Computer-Simulation-Using-Particles-Hockney/dp/0852743920>
  - b. Hermite integration:  
<http://articles.adsabs.harvard.edu//full/1992PASJ...44..141M/0000141.000.html>
  - c. FastPM: <https://github.com/fastpm/fastpm>
  - d. COLA: <https://bitbucket.org/tassev/colacode/src/hg/>

# Project-IV: Conjugate Gradient Poisson Solver

1. Implement the conjugate gradient method (CG) to solve the 2D Poisson equation
  - a. Measure the performance scaling (i.e., wall-clock time vs. number of cells)
  - b. Compare the performance with SOR
2. Bonus points
  - a. You get bonus points automatically since topic is not covered by our lecture

# Project-V: Divergence Cleaning for MHD

1. Implement a divergence cleaning method for 2D or 3D MHD (1D is trivial)
  - a. Compare the divergence errors with and without this correction
  - b. No need to adopt the constraint transport technique
  - c. Apply to a MHD test problem
  - d. Measure the performance scaling (i.e., wall-clock time vs. number of cells)
2. Bonus points
  - a. You get bonus points automatically since this topic is not covered by our lecture
  - b. Incorporate with a high-resolution shock capturing scheme
3. Reference
  - a. Hyperbolic Divergence Cleaning for the MHD Equations:  
<https://www.sciencedirect.com/science/article/pii/S002199910196961X>

# Project-VI: Tree Algorithm

1. **Implement a tree algorithm to compute gravity in 2D**
  - a. **Compare the performance and accuracy with the direct N-body method and with different cell-opening criteria**
  - b. **Measure the performance scaling (i.e., wall-clock time vs. number of particles)**
2. **Bonus points**
  - a. **You get bonus points automatically since this topic is not covered by our lecture**
  - b. **Extend to 3D**
  - c. **Orbit integration**
  - d. **Periodic boundary condition**
3. **Reference**
  - a. **The cosmological simulation code GADGET-2:**  
<https://www.mpa.mpa-garching.mpg.de/gadget/gadget2-paper.pdf>
  - b. **A hierarchical  $O(N \log N)$  force-calculation algorithm:**  
<https://www.nature.com/articles/324446a0>
  - c. **JZ-Tree: GPU friendly neighbour search and friends-of-friends with dual tree walks in JAX plus CUDA:**  
<https://arxiv.org/abs/2604.05885>

# Project-VII: Fast Multipole Method

1. Implement a Fast Multipole Method (FMM) to compute gravity in 2D
  - a. Compare the performance and accuracy with the direct N-body method
  - b. Measure the performance scaling (i.e., wall-clock time vs. number of particles)
2. Bonus points
  - a. You get bonus points automatically since this topic is not covered by our lecture
  - b. Extend to 3D
  - c. Orbit integration
  - d. Periodic boundary condition
3. Reference
  - a. The Fast Multipole Method: Numerical Implementation:  
<https://www.sciencedirect.com/science/article/pii/S0021999100964519>
  - b. A short course on fast multipole methods:  
[https://math.nyu.edu/faculty/greengar/shortcourse\\_fmm.pdf](https://math.nyu.edu/faculty/greengar/shortcourse_fmm.pdf)

# Project-VIII: Unstructured Moving Mesh

1. Implement a unstructured moving mesh method for hydrodynamics in 2D
  - a. Compare with a Cartesian fixed-grid method
  - b. Measure the performance scaling (i.e., wall-clock time vs. number of cells)
2. Bonus points
  - a. You get bonus points automatically since this topic is not covered by our lecture
  - b. Extend to 3D
  - c. Support refining and derefining cells
3. Reference
  - a. E pur si muove: Galileian-invariant cosmological hydrodynamical simulations on a moving mesh: <https://academic.oup.com/mnras/article/401/2/791/1147356>
  - b. The AREPO Public Code Release: <https://iopscience.iop.org/article/10.3847/1538-4365/ab908c>
  - c. Improving the convergence properties of the moving-mesh code AREPO: <https://academic.oup.com/mnras/article/455/1/1134/986779>
  - d. The moving mesh code Shadowfax: <https://www.sciencedirect.com/science/article/abs/pii/S2213133716300427>

# Project-IX: Smoothed Particle Hydrodynamics

1. Implement a smoothed particle hydrodynamics (SPH) method for hydrodynamics in 2D
  - a. Compare with a Cartesian fixed-grid method
  - b. Measure the performance scaling (i.e., wall-clock time vs. number of cells)
2. Bonus points
  - a. You get bonus points automatically since this topic is not covered by our lecture
  - b. Extend to 3D
  - c. Support MHD using a divergence cleaning method
3. Reference
  - a. Smoothed Particle Hydrodynamics in Astrophysics:  
<https://www.annualreviews.org/content/journals/10.1146/annurev-astro-081309-130914>
  - b. GADGET-4 code: <https://wwwmpa.mpa-garching.mpg.de/gadget4/>
  - c. SWIFT code: <https://swift.strw.leidenuniv.nl/>

# Project-X: Exact Riemann solver

1. Implement the exact Riemann solver in 1D
2. Compare the accuracy and performance with the Roe's solver from a set of Riemann problems. Especially, find a Riemann problem that the exact solver performs clearly better.
3. Bonus points
  - a. You get bonus points automatically since this topic is not covered by our lecture
  - b. Extend to 2D or even 3D
  - c. Support MHD
  - d. Add external and/or self-gravity
4. References
  - a. *Riemann Solvers and Numerical Methods for Fluid Dynamics -- A Practical Introduction*. Toro, Eleuterio F.
  - b. GAMER:  
[https://github.com/gamer-project/gamer/blob/master/src/Model\\_Hydro/CPU\\_Hydro/CPU\\_Shared\\_RiemannSolver\\_Exact.cpp](https://github.com/gamer-project/gamer/blob/master/src/Model_Hydro/CPU_Hydro/CPU_Shared_RiemannSolver_Exact.cpp)
  - c. Athena:  
<https://github.com/PrincetonUniversity/Athena-Cversion/blob/master/src/rsolvers/exact.c>

# Project-Y

1. **YOU DECIDE!!**
  - a. **Must still follow the “General Guidelines and Requirements”**
  - b. **Better discuss with me in advance**
  - c. **Better not to be directly related to your thesis (but reach me if you have a good idea!)**
2. **Bonus points**
  - a. **Apply to a real-world problem**

# References

- Git
  - [為你自已學 Git](#)
  - [GitHub tutorial](#)
  - [Git Will Finally Make Sense After This](#)